

# PROYECTO DE SEGMENTACIÓN DE IMÁGENES DE DRON COMO SEÑALES ESPACIALES

Carol Valentina Castellanos  
Jaimes  
*Estudiante de Ingeniería  
Electrónica*  
Universidad del Magdalena  
Santa Marta, Magdalena  
[cvcastellanos@unimagdalena.edu.co](mailto:cvcastellanos@unimagdalena.edu.co)

Fralllely Daniela Marquina  
Rodríguez  
*Estudiante de Ingeniería  
Electrónica*  
Universidad del Magdalena  
Santa Marta, Magdalena  
[fmarquina@unimagdalena.edu.co](mailto:fmarquina@unimagdalena.edu.co)

**Resumen-** En este proyecto se desarrolló un sistema de segmentación de imágenes RGB capturadas por dron para el conteo automático de piscinas de la granja experimental de la Universidad del Magdalena. La imagen fue tratada como una señal espacial discreta y procesada mediante conversión al espacio de color HSV, umbralización, operaciones morfológicas y el algoritmo Watershed. El método permitió detectar automáticamente las piscinas a partir de las características espectrales asociadas al agua. Los resultados obtenidos evidencian que las técnicas de procesamiento digital de imágenes son efectivas para la identificación y conteo de objetos en imágenes aéreas, aunque factores como sombras intensas y baja diferenciación espectral pueden afectar la precisión de la segmentación.

**Palabras Claves—** Segmentación de imágenes, procesamiento digital de imágenes, imágenes RGB, señales espaciales, espacio de color HSV, operaciones morfológicas, algoritmo Watershed, conteo de objetos.

**Abstract —** This project developed a segmentation system for RGB images captured by drones for the automatic counting of pools at the University of Magdalena's experimental farm. The image was treated as a discrete spatial signal and processed through conversion to the HSV color space, thresholding, morphological operations, and the Watershed algorithm. The method allowed for the automatic detection of pools based on the spectral characteristics associated with water. The results obtained demonstrate that digital image processing techniques are effective for identifying and counting objects in aerial images, although factors such as intense shadows and low spectral differentiation can affect the accuracy of the segmentation.

**Keywords —** Image segmentation, digital image processing, RGB images, spatial signals, HSV color space, morphological operations, Watershed algorithm, object counting.

## I. MARCO TEORICO

### A. Imágenes Digitales como Señales Espaciales

Una imagen digital puede interpretarse como una señal discreta bidimensional, donde cada píxel representa una muestra espacial de la escena observada. En una imagen RGB, cada píxel está compuesto por tres componentes de color: rojo (R), verde (G) y azul (B), que almacenan información de intensidad luminosa.

Matemáticamente, una imagen digital puede representarse como una función bidimensional:

$$I(x,y) \quad (1)$$

donde  $x$  e  $y$  corresponden a las coordenadas espaciales del píxel y  $I$  representa su intensidad o valor de color.

El procesamiento digital de imágenes aplica conceptos de procesamiento de señales para extraer información útil mediante técnicas de filtrado, segmentación y análisis espacial.

### B. Espacio de Color HSV

El espacio de color HSV (Hue, Saturation, Value) es una representación alternativa al modelo RGB que facilita la segmentación de objetos basada en características cromáticas.

Sus componentes son:

- Hue (H): Representa el tono o color dominante.

- Saturation (S): Indica el grado de pureza del color.
- Value (V): Corresponde al brillo o intensidad luminosa.

La conversión de RGB a HSV permite separar la información de color de la iluminación, facilitando la identificación de regiones asociadas al agua presentes en las piscinas.

### C. Segmentación de Imágenes

La segmentación es el proceso mediante el cual una imagen se divide en regiones homogéneas con características similares de color, textura o intensidad.

Una técnica común consiste en la umbralización, donde se seleccionan rangos específicos de valores para clasificar cada píxel como perteneciente o no al objeto de interés. El resultado es una máscara binaria definida como:

$$M(x,y) = \{1 \text{ si el píxel pertenece al objeto}; 0 \text{ si en otro caso} \} \quad (2)$$

En este proyecto la segmentación se empleó para identificar las regiones correspondientes a las piscinas dentro de la imagen aérea.

### D. Operaciones Morfológicas

Las operaciones morfológicas son técnicas de procesamiento espacial utilizadas para mejorar las máscaras binarias obtenidas durante la segmentación.

Entre las más utilizadas se encuentran:

- Erosión: Elimina pequeñas regiones de ruido.
- Dilatación: Expande las regiones detectadas.
- Apertura: Reduce objetos pequeños y suaviza contornos.
- Cierre: Rellena huecos dentro de los objetos segmentados.

Estas operaciones permiten mejorar la forma de las regiones detectadas y aumentar la precisión del conteo de objetos.

### E. Algoritmo Watershed

Es una técnica de segmentación basada en la interpretación topográfica de la imagen. Considera la imagen como una superficie tridimensional donde los valores de intensidad representan elevaciones.

El método identifica cuencas de captación y líneas divisorias para separar regiones conectadas que pertenecen a objetos distintos.

Su principal ventaja es la capacidad de dividir objetos cercanos o parcialmente unidos, permitiendo identificar cada piscina como una instancia independiente para realizar el conteo automático.

### F. Conteo de Objetos Mediante Componentes Conectados

Una vez obtenida la máscara segmentada, el conteo de objetos puede realizarse mediante análisis de componentes conectados. Este procedimiento agrupa píxeles vecinos pertenecientes a una misma región y asigna una etiqueta única a cada objeto detectado.

Si  $N$  representa el número total de regiones identificadas, el conteo de objetos se expresa como:

$$N = \sum_{i=1}^k R_i \quad (3)$$

donde  $R_i$  corresponde a cada región conectada detectada en la imagen.

Esta técnica permite estimar automáticamente el número de piscinas presentes en la escena y evaluar el desempeño del método de segmentación.

### G. Factores que Afectan la Segmentación

La precisión de los resultados puede verse afectada por diversos factores presentes en las imágenes capturadas por dron:

- Sombras proyectadas por estructuras o vegetación.
- Variaciones de iluminación.
- Reflejos sobre la superficie del agua.
- Baja resolución espacial.
- Similitud espectral entre objetos diferentes.

Estas condiciones pueden ocasionar errores de detección o la pérdida parcial de algunas regiones de interés, afectando el conteo final de las piscinas.

## II. DESARROLLO

### A. Modalidad RGB justificada

Se eligió trabajar con imágenes RGB porque el reto de conteo de piscinas se resuelve eficientemente mediante características visuales directas como color, forma y textura, sin necesidad de información espectral adicional. Las piscinas presentan tonos azulados-verdosos del agua fácilmente discriminables mediante segmentación HSV y operaciones morfológicas, técnicas plenamente aplicables sobre los tres canales RGB.

## B. Inserción de datos

- Código implementado

Se utilizó la función `files.upload()` de Google Colab para importar la imagen aérea capturada por dron al entorno de trabajo. Una vez cargado el archivo, este quedó disponible para su procesamiento mediante las librerías de visión por computador y análisis de imágenes.

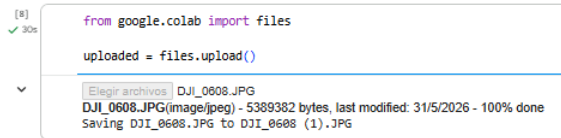


Figura 1. Código de carga de imagen

Se empleó la librería OpenCV (`cv2`) para leer la imagen seleccionada. Debido a que OpenCV almacena las imágenes en formato BGR, se realizó una conversión al espacio RGB mediante la función `cv2.cvtColor()` con el fin de preservar la representación correcta de los colores. Posteriormente, utilizando Matplotlib, se visualizó la imagen original para inspeccionar las características de la escena, identificar las piscinas presentes y verificar la calidad de los datos antes de iniciar las etapas de segmentación y conteo automático.

### Lectura y visualización

Se utilizó OpenCV (`cv2`) para el análisis de la imagen. Debido a que OpenCV carga las imágenes en formato BGR, fue necesario convertirlas a RGB para mantener la representación correcta de los colores durante el procesamiento.

```
import cv2
import matplotlib.pyplot as plt

# Obtener nombre del archivo subido
nombre_imagen = list(uploaded.keys())[0]

# Leer imagen
img_bgr = cv2.imread(nombre_imagen)

# Convertir BGR -> RGB
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

# Mostrar imagen
plt.figure(figsize=(10,6))
plt.imshow(img_rgb)
plt.title("Imagen Original")
plt.axis("off")
plt.show()
```

Figura 2. Código de lectura y visualización de la imagen

- Descripción de imagen

La imagen original corresponde a una fotografía aérea RGB capturada por un dron sobre la granja experimental de la Universidad del Magdalena. En ella se observan diez piscinas rectangulares distribuidas en dos filas y separadas por caminos de tierra. Las piscinas presentan diferentes tonalidades debido a variaciones en la cantidad de agua, sedimentos y condiciones de iluminación. Estas diferencias visuales, junto con la presencia de sombras y reflejos, representan desafíos para la segmentación automática, ya que pueden afectar la identificación precisa de las regiones correspondientes a las piscinas.

Sin embargo, la vista cenital de la imagen facilita la aplicación.



Figura 3. Imagen original tomada por el dron

## C. Imagen como señal discreta

- Código implementado

Ilustra el concepto de imagen como señal espacial discreta usando NumPy. Primero imprime las dimensiones generales de la imagen con `img_rgb.shape` y el tipo de dato de sus píxeles con `img_rgb.dtype`, luego extrae una región de 5x5 píxeles mediante slicing (`img_rgb[100:105, 100:105]`) y recorre sus filas con un bucle `for` para mostrar los valores RGB discretos de cada píxel. Finalmente, repite el proceso sobre el canal rojo aislado. R, imprimiendo su forma con `R.shape` y sus valores numéricos en la misma región, demostrando que cada píxel no es más que un conjunto de números enteros en una cuadrícula bidimensional.

```
import numpy as np

# Asumiendo que 'img_rgb' ya está cargada y es la imagen completa en formato RGB

print(f"Dimensiones de la imagen (alto, ancho, canales): {img_rgb.shape}")
print(f"Tipo de datos de los píxeles: {img_rgb.dtype}")

# Seleccionar una pequeña región de la imagen (por ejemplo, 10x10 píxeles)
# Esto nos permite ver los valores numéricos discretos de los píxeles
region_ejemplo = img_rgb[100:105, 100:105] # Selecciona una región de 5x5 píxeles

print("\nValores de píxeles (RGB) para una región de 5x5:\n")
for row in region_ejemplo:
    print(row)

print("\nObserva que cada elemento en el array representa un píxel en una ubicación específica")

# También podemos ver un solo canal para ilustrar aún más la idea de una señal 2D
# Asumiendo que 'R' (canal rojo) ya ha sido extraído
print(f"Dimensiones del canal Rojo (alto, ancho): {R.shape}")
print(f"Valores de píxeles para una región de 5x5 en el canal Rojo:\n")
print(R[100:105, 100:105])
```

Figura 4. Código imagen discreta

- Análisis de resultados

Los resultados confirman que la imagen tiene dimensiones de 720×1280 píxeles con 3 canales (RGB) y valores enteros sin signo (`uint8`), lo que significa que cada píxel se representa con números discretos entre 0 y 255.

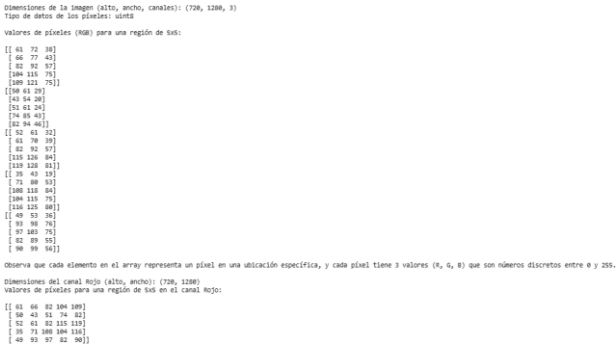


Figura 5. Datos en vector [R, G, B]

Al inspeccionar la región de 5×5 extraída, se observan tripletas como [61, 72, 38] o [104, 115, 75] donde cada valor corresponde a la intensidad de los canales R, G y B respectivamente, mostrando tonos verdosos-apagados consistentes con vegetación o terreno. El canal rojo aislado para la misma región presenta valores como [61, 50, 52, 35, 49] por fila, confirmando que cada canal es independientemente una matriz 2D de 720×1280 con un único valor numérico por píxel.

#### D. Características de la imagen

- Código implementado

Para la resolución, extrae las dimensiones con `img_rgb.shape` e imprime alto, ancho y canales. Para el análisis de sombras, convierte la imagen a HSV con `cv2.cvtColor(img_rgb, cv2.COLOR_RGB2HSV)`, extrae el canal V (brillo) con slicing `[:, :, 2]`, y genera una máscara booleana `shadow_mask = value_channel < 50` que identifica píxeles oscuros como sombras, calculando además el brillo promedio con `np.mean` y el porcentaje de sombra con `np.sum`.

Características de la imagen: resolución, análisis de sombras, ruido y perspectiva

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Asegúrate de que 'img_rgb' esté disponible de celdas anteriores

print("--- 1. Análisis de Resolución ---")
height, width, channels = img_rgb.shape
print(f"La imagen original tiene una resolución de (width)x(height) píxeles.")
print(f"Cada píxel contiene (channels) canales de color (R, G, B).")

print("\n--- 2. Análisis de Sombras ---")
# Convertir la imagen completa a HSV para analizar el canal V (Brillo)
img_hsv = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2HSV)
value_channel = img_hsv[:, :, 2] # extraer el canal de Valor (Brillo)

# Definir un umbral para identificar regiones oscuras (sombras)
# Este umbral puede necesitar ajuste dependiendo de la imagen
shadow_threshold = 50 # valores de V por debajo de 50 se consideran sombras
shadow_mask = value_channel < shadow_threshold

plt.figure(figsize=(15, 6))

plt.subplot(1, 3, 1)
plt.imshow(img_rgb)
plt.title("Imagen Original")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(value_channel, cmap="gray")
plt.title("Canal V (Brillo)")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(shadow_mask, cmap="gray")
plt.title("Regiones de Sombra (V < {shadow_threshold})")
plt.axis("off")
plt.show()

print("Las regiones en blanco en la última imagen indican posibles sombras.")
print(f"Valores de brillo (V) promedio en la imagen: (np.mean(value_channel)).zf)")
print(f"Porcentaje de píxeles en sombra: ((np.sum(shadow_mask) / (height * width)) * 100:zf)")

print("\n--- 3. Análisis de Ruido ---")
# una forma simple de estimar el ruido es calcular la desviación estándar de la intensidad de los píxeles.
# en una imagen idealmente "plana" o sin ruido, la desviación estándar sería baja. Aquí, calculamos
# la desviación estándar para cada canal. Una desviación estándar alta puede indicar textura o ruido.
```

Figura 6. Código análisis de resolución y sombras

Para el análisis de ruido, calcula la desviación estándar por canal con `np.std` sobre cada capa RGB, luego aplica un suavizado gaussiano con `cv2.GaussianBlur(img_rgb, (5,5), 0)` y obtiene la diferencia absoluta con `cv2.absdiff` entre la imagen original y la suavizada, visualizando así las zonas de mayor ruido o detalle fino. Finalmente, para el análisis de perspectiva, superpone líneas guía verticales y horizontales con `plt.axvline` y `plt.axhline` en posiciones proporcionales al 25% y 75% del ancho y alto, simulando una cuadrícula que ayuda a evaluar visualmente la distorsión y convergencia de la imagen capturada desde dron.

```
std_dev_R = np.std(img_rgb[:, :, 0])
std_dev_G = np.std(img_rgb[:, :, 1])
std_dev_B = np.std(img_rgb[:, :, 2])

print(f"Desviación estándar de los píxeles (Canal R): (std_dev_R:zf)")
print(f"Desviación estándar de los píxeles (Canal G): (std_dev_G:zf)")
print(f"Desviación estándar de los píxeles (Canal B): (std_dev_B:zf)")

# Para una estimación visual del ruido, podemos comparar la imagen original
# con una versión ligeramente suavizada. Las diferencias pueden acentuar el ruido.
blurred_img = cv2.GaussianBlur(img_rgb, (5, 5), 0)
noise_visualization = cv2.absdiff(img_rgb, blurred_img)

plt.figure(figsize=(15, 5))
plt.subplot(1, 3, 1)
plt.imshow(img_rgb)
plt.title("Imagen Original")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(blurred_img)
plt.title("Imagen Suavizada (gaussianblur)")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(noise_visualization, cmap="gray")
plt.title("Visualización de Ruido (diferencia)")
plt.axis("off")
plt.show()

print("Las áreas más claras en la imagen de 'Diferencia' pueden indicar la presencia de ruido o detalles")

print("\n--- 4. Análisis de Perspectiva ---")
# la perspectiva se refiere a cómo los objetos en una escena parecen cambiar de tamaño
# y forma a medida que se alejan del observador (cámara). En imágenes de drones,
# la perspectiva a menudo implica objetos que se vuelven más pequeños y líneas paralelas
# que convergen en puntos de fuga, aunque en una vista aérea cenital o casi cenital,
# este efecto puede ser menos pronunciado.

plt.figure(figsize=(10, 8))
plt.imshow(img_rgb)
plt.title("Análisis de Perspectiva en la Imagen Original")
plt.axis("off")

# Añadir algunas líneas de guía para ilustrar la perspectiva (ejemplo)
# Estas líneas son solo ilustrativas y podrían no coincidir perfectamente con la imagen
# dibujar líneas para simular una cuadrícula que converge
plt.axvline(x=width*0.25, color='red', linestyle='--', linewidth=1, alpha=0.7)
plt.axvline(x=width*0.75, color='red', linestyle='--', linewidth=1, alpha=0.7)
plt.axhline(y=height*0.25, color='blue', linestyle='--', linewidth=1, alpha=0.7)
plt.axhline(y=height*0.75, color='blue', linestyle='--', linewidth=1, alpha=0.7)

print("En esta imagen, la perspectiva se observa en cómo los objetos más distantes (como las piscinas más
print("La imagen fue tomada desde un dron, lo que generalmente minimiza la distorsión de perspectiva en :
plt.show()
```

Figura 7. Código análisis de ruido y perspectiva

- Análisis de resultados

La imagen de 1280×720 píxeles con 3 canales RGB presenta un brillo promedio de 124.21 sobre 255, indicando una iluminación moderada. La máscara de sombras (`V < 50`) muestra que apenas el 4.84% de los píxeles están en sombra, concentradas principalmente en la vegetación y bordes de las piscinas, lo que significa que la imagen tiene buenas condiciones de iluminación general para el análisis.

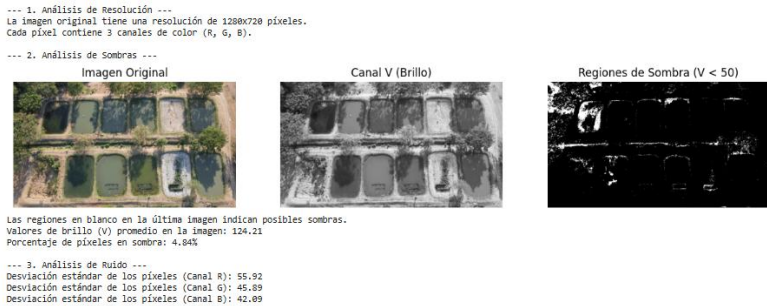


Figura 8. Resolución y sombras

En cuanto al ruido, las desviaciones estándar por canal (R: 55.92, G: 45.89, B: 42.09) son relativamente altas, lo que refleja la gran variedad de texturas presentes en la escena (agua, tierra, vegetación) más que ruido real del sensor. La visualización de diferencia entre la imagen original y la suavizada con GaussianBlur aparece casi completamente negra, confirmando que el ruido de alta frecuencia es mínimo. El análisis de perspectiva muestra que, al ser una toma cenital de dron, la distorsión es leve y se limita a los bordes, donde las piscinas más lejanas aparecen ligeramente más pequeñas.



Figura 9. Ruido y perspectiva

## E. Separación de canales RGB

- Código implementado

Se realizó la separación de una imagen RGB en sus tres canales de color: rojo (R), verde (G) y azul (B), almacenando cada componente en una matriz independiente. Posteriormente, se mostraron las dimensiones de cada canal mediante la propiedad shape, con el fin de verificar que conservan la misma resolución de la imagen original y comprobar que la separación se realizó correctamente.

### 🔥 Separación de canales RGB

La imagen RGB puede descomponerse en tres matrices bidimensionales correspondientes a los canales rojo (R), verde (G) y azul (B). Cada canal representa una componente de intensidad de la señal espacial.

```
# Separar canales RGB
R = img_rgb[:, :, 0]
G = img_rgb[:, :, 1]
B = img_rgb[:, :, 2]

# Mostrar dimensiones
print("Dimensión canal R:", R.shape)
print("Dimensión canal G:", G.shape)
print("Dimensión canal B:", B.shape)

# Mostrar canales
plt.figure(figsize=(15,5))

plt.subplot(1,3,1)
plt.imshow(R, cmap='gray')
plt.title("Canal R")
plt.axis("off")

plt.subplot(1,3,2)
plt.imshow(G, cmap='gray')
plt.title("Canal G")
plt.axis("off")

plt.subplot(1,3,3)
plt.imshow(B, cmap='gray')
plt.title("Canal B")
plt.axis("off")

plt.show()
```

Figura 10. Código separación de canales

Para visualizar los resultados, se utilizó la librería Matplotlib, generando una figura con tres subgráficas donde se representa cada canal por separado en escala de grises. Esto permite observar la distribución de intensidad de cada componente de color y analizar su contribución en la formación de la imagen original. Finalmente, se ocultaron los ejes para mejorar la presentación visual y se mostraron las imágenes mediante la función plt.show().

- Análisis de resultados

Las dimensiones obtenidas para los canales R, G y B fueron (720, 1280), lo que significa que cada canal está representado por una matriz de 720 filas y 1280 columnas, correspondientes al alto y ancho de la imagen en píxeles. El hecho de que los tres canales tengan las mismas dimensiones confirma que la separación de los componentes rojo, verde y azul se realizó correctamente, conservando la resolución espacial original de la imagen y permitiendo analizar individualmente la intensidad de cada color en cada píxel.

```
Dimensión canal R: (720, 1280)
Dimensión canal G: (720, 1280)
Dimensión canal B: (720, 1280)
```

Figura 11. Dimensiones obtenidas

- Imágenes generadas

Las imágenes correspondientes a los canales R, G y B muestran la descomposición de la imagen original en sus componentes de color rojo, verde y azul, representadas en escala de grises. En los tres canales se conserva claramente la estructura de la escena, que corresponde a una vista aérea de lo que parecen ser estanques o piscinas rectangulares rodeadas de vegetación. Se observan diferencias sutiles de brillo entre los canales: el canal R presenta zonas ligeramente más claras en las áreas de suelo y bordes de los estanques, el canal G muestra una distribución de intensidad intermedia y bastante homogénea, mientras que el canal B tiende a presentar las regiones de agua y sombra con tonos algo más oscuros, lo que refleja la menor reflectancia azul de estos elementos naturales. Estas variaciones confirman que cada canal aporta información cromática distinta a la composición final de la imagen.

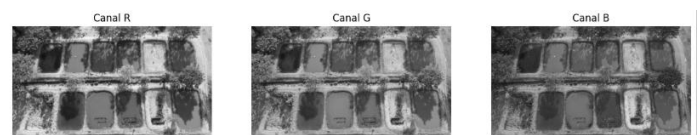


Figura 12. Imágenes de los canales R, G y B

## F. Análisis estadístico

- Código implementado

Se utilizó la librería NumPy para realizar un análisis estadístico básico de los canales RGB previamente obtenidos. Mediante la función `np.mean()`, se calculó el valor promedio de intensidad de los canales rojo (R), verde (G) y azul (B), lo que permite conocer el nivel medio de brillo presente en cada componente de color de la imagen.

### ☀ Análisis estadístico de los canales RGB

Aunque los canales RGB pueden verse similares en escala de grises, sus valores numéricos son diferentes. Por esta razón se calculan estadísticas básicas de intensidad para comparar la respuesta de cada canal.

```
import numpy as np
#numpy es una librería que maneja matrices y operaciones matematicas
print("Promedio canal R:", np.mean(R))
print("Promedio canal G:", np.mean(G))
print("Promedio canal B:", np.mean(B))

print("Mínimo R:", np.min(R), "Máximo R:", np.max(R))
print("Mínimo G:", np.min(G), "Máximo G:", np.max(G))
print("Mínimo B:", np.min(B), "Máximo B:", np.max(B))
```

Figura 13. Código del análisis estadístico

Además, se emplearon las funciones `np.min()` y `np.max()` para determinar los valores mínimos y máximos de intensidad en cada canal. Estos parámetros permiten identificar el rango dinámico de los colores presentes en la imagen y comparar la distribución de intensidades entre los canales RGB, proporcionando una descripción cuantitativa complementaria al análisis visual realizado anteriormente.

- Análisis de resultados

Los resultados obtenidos muestran que el canal rojo (118.86) y el canal verde (116.21) poseen intensidades promedio muy similares, mientras que el canal azul (98.34) presenta un valor promedio considerablemente menor. Esto indica que en la imagen predominan los tonos asociados a los componentes rojo y verde, mientras que la contribución del azul es más reducida. Además, los tres canales presentan un valor mínimo de 0 y un máximo de 255, lo que confirma que se utiliza todo el rango de intensidades disponible en una imagen de 8 bits. Estos resultados evidencian una distribución desigual de los colores en la escena, información que puede ser útil para tareas de procesamiento digital de imágenes.

```
Promedio canal R: 118.8604606944444
Promedio canal G: 116.20692708333333
Promedio canal B: 98.34719075520833
Mínimo R: 0 Máximo R: 255
Mínimo G: 0 Máximo G: 255
Mínimo B: 0 Máximo B: 255
```

Los valores promedio de intensidad muestran diferencias entre los canales RGB. El canal azul presenta una intensidad promedio respecto a los canales rojo y verde, lo cual indica una respuesta distinta de las superficies de agua y vegetación presentes en la imagen. Estas diferencias permiten utilizar la información espectral de los canales para tareas de segmentación.

Figura 14. Datos estadísticos obtenidos

## G. Recorte de región de interés

- Código implementado

Se definió una región de interés (ROI) mediante un recorte de la imagen original (`img_rgb[80:680, 120:1180]`), seleccionando únicamente el área de las piscinas para reducir falsas detecciones causadas por árboles, sombras u otros elementos externos. Luego, la región recortada se almacenó en `roi` y se visualizó con Matplotlib, configurando el tamaño de la figura, agregando un título y ocultando los ejes para facilitar su análisis.

### Recorte de región de interés

Para reducir falsas detecciones en árboles y sombras, se seleccionó una región de interés que contiene únicamente el área de las piscinas.

```
# Recortar región de interés aproximada
# Formato: imagen[y1:y2, x1:x2]
roi = img_rgb[80:680, 120:1180]

plt.figure(figsize=(12,6))
plt.imshow(roi)
plt.title("Región de interés - piscinas")
plt.axis("off")
plt.show()
```

Figura 15. Código de recorte

- Descripción de imagen

La imagen muestra una región de interés (ROI) titulada "Región de interés - piscinas", correspondiente a una vista aérea de un conjunto de estanques o piscinas de acuicultura distribuidos en dos filas. Se identifican un total de siete estanques rectangulares de esquinas redondeadas, con diferentes niveles y coloraciones de agua que van desde tonos verdes oscuros hasta grises claros, lo que podría indicar distintas condiciones del agua o etapas de uso. Los estanques están separados por caminos de tierra y rodeados de vegetación. Este recorte es útil porque concentra el análisis únicamente en el área de interés, reduciendo la presencia de elementos externos irrelevantes y mejorando la precisión de los algoritmos de visión por computadora encargados de identificar, segmentar o analizar los cuerpos de agua.



Figura 16. Región de interés

## H. Conversión de la región de interés a HSV

- Código implementado

Se convirtió la región de interés (ROI) del espacio de color RGB a HSV mediante la función `cv2.cvtColor()`, con el objetivo de resaltar mejor las diferencias entre las piscinas y el entorno circundante. Posteriormente, se definieron rangos mínimos (lower) y máximos (upper) en HSV para identificar los tonos asociados al agua y se generó una máscara binaria utilizando `cv2.inRange()`, donde los píxeles dentro del rango seleccionado se destacan del resto de la imagen. Finalmente, la máscara obtenida se visualizó con Matplotlib en escala de grises, facilitando la identificación de las áreas de interés para su posterior análisis.

#### # Conversión de la región de interés a HSV

La región de interés fue convertida al espacio HSV para resaltar las diferencias entre las piscinas y el entorno cercano.

```
# Convertir ROI a HSV
roi_hsv = cv2.cvtColor(roi, cv2.COLOR_RGB2HSV)

# Nuevos rangos HSV
lower = np.array([35, 20, 20])
upper = np.array([100, 200, 150])

# Crear máscara
mask_roi = cv2.inRange(roi_hsv, lower, upper)

# Mostrar máscara
plt.figure(figsize=(12,6))
plt.imshow(mask_roi, cmap='gray')
plt.title("Máscara")
plt.axis("off")
plt.show()
```

Figura 17. Código de conversión a HSV

- Descripción de imagen

La imagen corresponde a una máscara binaria, donde las zonas blancas representan los píxeles detectados como agua o superficie de los estanques, y las zonas negras corresponden a los bordes, caminos y vegetación que fueron descartados. Se puede apreciar que la máscara logra identificar de forma general las siete piscinas presentes en la escena, conservando su forma rectangular con esquinas redondeadas. Sin embargo, se observa una cantidad considerable de ruido al interior de los estanques, manifestado como píxeles negros dispersos dentro de las regiones blancas, así como fragmentos blancos irregulares en zonas que corresponden a los bordes y separaciones entre piscinas.



Figura 18. Imagen en HSV

- Código implementado

El código implementa un sistema de detección y conteo automático de piscinas a partir de la máscara binaria, utilizando técnicas de visión por computadora con OpenCV. En la primera etapa se realiza una limpieza morfológica de la máscara mediante una operación de apertura (`cv2.MORPH_OPEN`) con un kernel de  $9 \times 9$  aplicada dos veces, lo que permite eliminar el ruido pequeño y separar regiones que aparecían unidas.

Se aplica `cv2.connectedComponentsWithStats` para identificar los componentes conectados de la máscara limpia, extrayendo para cada uno su posición, dimensiones y área. Cada componente es filtrado mediante criterios geométricos: área mayor a 1800 píxeles, ancho y alto superiores a 35 píxeles, una relación de aspecto entre 0.35 y 3.2, y un factor de relleno mayor a 0.25, con el objetivo de conservar únicamente las regiones que corresponden a piscinas reales.

#### ☀ Separación de regiones mediante Watershed

Debido a que algunas piscinas aparecían unidas dentro de la máscara binaria, se utilizó el algoritmo Watershed para separar regiones conectadas y mejorar el conteo automático.

```
resultado = roi.copy()
boxes = []

for label in np.unique(markers):
    if label <= 1:
        continue

    region = np.uint8(markers == label) * 255
    contours, _ = cv2.findContours(region, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    for cnt in contours:
        area = cv2.contourArea(cnt)
        x, y, w, h = cv2.boundingRect(cnt)
        ratio = w / h

        if area > 3500 and w > 60 and h > 60 and 0.5 < ratio < 2.5:
            boxes.append((x, y, w, h))

# Eliminar cajas muy superpuestas
boxes_filtradas = []

for box in boxes:
    x, y, w, h = box
    nueva = True

    for bx, by, bw, bh in boxes_filtradas:
        inter_x1 = max(x, bx)
        inter_x2 = min(x+w, bx+bw)
        inter_y1 = min(y, by)
        inter_y2 = min(y+h, by+bh)

        inter_area = max(0, inter_x2-inter_x1) * max(0, inter_y2-inter_y1)
        area_box = w * h
        area_b = bw * bh

        overlap = inter_area / min(area_box, area_b)
```

Figura 19. Sistema de detección

Después se realiza un filtrado de cajas duplicadas o muy solapadas, recorriendo las detecciones y calculando el área de intersección entre cada par de cajas; si el solapamiento supera el 25% respecto a la caja más pequeña, la detección redundante es descartada. Las cajas filtradas se ordenan por posición y se dibujan sobre la imagen original con rectángulos rojos usando `cv2.rectangle`. Finalmente, el resultado se visualiza con Matplotlib, mostrando en el título el conteo automático de piscinas detectadas.

## I. Separación de regiones mediante Watershed

```

boxes.append((x, y, w, h))

# =====
# 3. ELIMINAR CAJAS MUY PARECIDAS
# =====

boxes_filtradas = []

for box in boxes:
    x, y, w, h = box
    nueva = True

    for bx, by, bw, bh in boxes_filtradas:

        inter_x1 = max(x, bx)
        inter_y1 = max(y, by)
        inter_x2 = min(x + w, bx + bw)
        inter_y2 = min(y + h, by + bh)

        inter_area = max(0, inter_x2 - inter_x1) * max(0, inter_y2 - inter_y1)

        area_box = w * h
        area_b = bw * bh

        overlap = inter_area / min(area_box, area_b)

        if overlap > 0.25:
            nueva = False
            break

    if nueva:
        boxes_filtradas.append(box)

# =====
# 4. DIBUJAR RESULTADO SOBRE LA IMAGEN ORIGINAL
# =====

boxes_filtradas = sorted(boxes_filtradas, key=lambda b: (b[1], b[0]))

for x, y, w, h in boxes_filtradas:
    cv2.rectangle(resultado, (x, y), (x + w, y + h), (255, 0, 0), 3)

contador = len(boxes_filtradas)

plt.figure(figsize=(12,6))
plt.imshow(resultado)
plt.title(f"Conteo automático de piscinas: {contador}")
plt.axis("off")
plt.show()

print("Número de piscinas detectadas automáticamente:", contador)

```

Figura 20. Código para las cajas

## • Descripción de imagen

La imagen muestra el resultado final del algoritmo, donde se detectaron y delimitaron automáticamente 9 piscinas mediante rectángulos rojos sobre la imagen original. Si bien la mayoría de los estanques fueron identificados correctamente, algunas cajas delimitadoras presentan imprecisiones en su tamaño o posición, lo que indica que el sistema podría beneficiarse de un ajuste fino en los umbrales geométricos para mejorar la precisión de las detecciones.



Figura 21. Detección de piscinas

## • Código implementado

Se implementa la función `analyze_sensitivity`, que reconstruye la máscara binaria con `cv2.inRange` usando los límites HSV recibidos, la limpia con `cv2.morphologyEx(MORPH_OPEN)` para eliminar ruido pequeño y separar regiones pegadas, luego detecta componentes conectados con `cv2.connectedComponentsWithStats` filtrando por área mínima, y finalmente dibuja los rectángulos sobre la ROI con `cv2.rectangle`, mostrando en tres subgráficas la máscara inicial, la máscara limpia y el conteo resultante.

### Análisis de sensibilidad de parámetros

```

import ipywidgets as widgets
from IPython.display import display
import matplotlib.pyplot as plt
import cv2
import numpy as np

def analyze_sensitivity(h_min, s_min, v_min, h_max, s_max, v_max, kernel_size_val, iterations_val, min_area_val):
    # Reconstruct HSV bounds
    lower_bound = np.array([h_min, s_min, v_min])
    upper_bound = np.array([h_max, s_max, v_max])

    # 1. Initial segmentation using HSV thresholds
    mask_roi_current = cv2.inRange(roi_hsv, lower_bound, upper_bound)

    # 2. Mask cleaning (morphological operations)
    # Ensure kernel_size_val is odd
    kernel_size_val = kernel_size_val if kernel_size_val % 2 != 0 else kernel_size_val + 1
    kernel_current = np.ones((kernel_size_val, kernel_size_val), np.uint8)
    mask_limpiar_current = cv2.morphologyEx(mask_roi_current, cv2.MORPH_OPEN, kernel_current, iterations=iterations_val)

    # 3. Connected components
    num_labels, labels, stats, centroids = cv2.connectedComponentsWithStats(mask_limpiar_current)

    boxes_current = []
    for i in range(1, num_labels):
        x = stats[i, cv2.CC_STAT_LEFT]
        y = stats[i, cv2.CC_STAT_TOP]
        w = stats[i, cv2.CC_STAT_WIDTH]
        h = stats[i, cv2.CC_STAT_HEIGHT]
        area = stats[i, cv2.CC_STAT_AREA]

        # Filter based on minimum area. For a more comprehensive analysis, ratio and fill could also be added.
        if area > min_area_val:
            boxes_current.append((x, y, w, h))

    # For simplicity, the filtering of very similar boxes (overlap) is omitted in this sensitivity analysis.
    # The boxes are just sorted for consistent display.
    boxes_filtradas_current = sorted(boxes_current, key=lambda b: (b[1], b[0]))

    # 4. Draw results on the original ROI image
    resultado_current = roi.copy()
    for x, y, w, h in boxes_filtradas_current:
        cv2.rectangle(resultado_current, (x, y), (x + w, y + h), (255, 0, 0), 3)

    contador_current = len(boxes_filtradas_current)

    # Display results
    fig, axes = plt.subplots(1, 3, figsize=(20, 7))

```

Figura 22. Código para las imágenes

Posteriormente, se define nueve `IntSlider` de `ipywidgets` con los rangos y valores base del proyecto para cada parámetro HSV, tamaño de kernel, iteraciones morfológicas y área mínima, los conecta a `analyze_sensitivity` mediante `widgets.interactive` y los despliega con `display`, logrando que cada vez que el usuario mueve un slider se reejecute automáticamente la segmentación completa y se actualicen las tres visualizaciones en tiempo real.

```

axes[0].imshow(mask_roi_current, cmap='gray')
axes[0].set_title("Máscara HSV inicial")
axes[0].axis("off")

axes[1].imshow(mask_limpia_current, cmap='gray')
axes[1].set_title("Máscara limpia (morfológica)")
axes[1].axis("off")

axes[2].imshow(resultado_current)
axes[2].set_title("Conteo automático de piscinas: (contador_current)")
axes[2].axis("off")

plt.tight_layout()
plt.show()

print(f"Número de piscinas detectadas: {contador_current}")
print(f"Parámetros usados: H_min={h_min}, S_min={s_min}, V_min={v_min}, H_max={h_max}, S_max={s_max}, V_max={v_max}, kernel_size={kernel_size}, iterations={iterations}, min_area={min_area}")

# Create widgets for parameters with initial values from the notebook
h_min_slider = widgets.IntSlider(min=0, max=179, step=1, value=35, description="H Min")
s_min_slider = widgets.IntSlider(min=0, max=255, step=1, value=20, description="S Min")
v_min_slider = widgets.IntSlider(min=0, max=255, step=1, value=20, description="V Min")
h_max_slider = widgets.IntSlider(min=0, max=179, step=1, value=100, description="H Max")
s_max_slider = widgets.IntSlider(min=0, max=255, step=1, value=200, description="S Max")
v_max_slider = widgets.IntSlider(min=0, max=255, step=1, value=150, description="V Max")
kernel_size_slider = widgets.IntSlider(min=1, max=21, step=2, value=9, description="Kernel Size") # Kernel size should be odd
iterations_slider = widgets.IntSlider(min=1, max=50, step=1, value=2, description="Iterations")
min_area_slider = widgets.IntSlider(min=100, max=5000, step=50, value=1800, description="Min Area")

# Link widgets to the function
sensitivity_widget = widgets.interactive(
    analyze_sensitivity,
    h_min=h_min_slider, s_min=s_min_slider, v_min=v_min_slider,
    h_max=h_max_slider, s_max=s_max_slider, v_max=v_max_slider,
    kernel_size_val=kernel_size_slider,
    iterations_val=iterations_slider,
    min_area_val=min_area_slider
)

display(sensitivity_widget)

```

Figura 23. Código para generar los sliders

- Descripción de imagen

Las tres imágenes muestran el funcionamiento de una herramienta interactiva para la detección automática de piscinas usando visión por computadora.

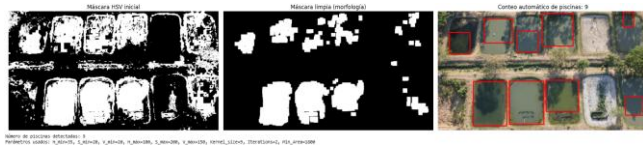


Figura 24. imágenes para controlar

El panel de sliders expone nueve parámetros ajustables en tiempo real: los valores mínimos y máximos de los canales H (matiz), S (saturación) y V (brillo) definen el rango de color HSV que el sistema considera como piscina, es decir, filtran los píxeles por su tono, intensidad de color y luminosidad respectivamente; el Kernel Size determina el tamaño de la ventana usada en las operaciones morfológicas para eliminar ruido; las Iterations controlan cuántas veces se aplica esa operación, logrando una limpieza más o menos agresiva; y el Min Area establece el umbral mínimo de píxeles que debe tener un componente conectado para ser contado como piscina, descartando así pequeñas regiones residuales.

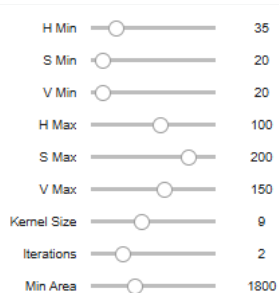


Figura 25. Sliders definidas

El resultado de combinar estos parámetros se visualiza en las tres etapas: la máscara HSV inicial muestra todos los píxeles que caen dentro del rango de color configurado, incluyendo ruido y falsas detecciones; la máscara limpia refleja el efecto del kernel y las iteraciones morfológicas, conservando solo regiones coherentes; y el panel final superpone bounding boxes rojos sobre la imagen original marcando únicamente los componentes que superaron el área mínima.

## K. Validación cuantitativa

- Código implementado

Se realizó una validación cuantitativa para evaluar la precisión del algoritmo de detección. Se definieron los valores correspondientes al conteo real y al conteo automático obtenido por el sistema. Posteriormente, se utilizó la función abs(), cuya finalidad es calcular el valor absoluto de la diferencia entre ambos conteos, permitiendo obtener el error absoluto sin considerar si el algoritmo detectó más o menos elementos de los reales. Se calculó el error porcentual, el cual sirve para expresar el error en relación con el número real de elementos detectados y así medir la exactitud del método de manera más representativa. Mediante la función print() se mostraron en pantalla el conteo manual, el conteo automático, el error absoluto y el error porcentual, facilitando la interpretación de los resultados.

### Validación cuantitativa

Se comparó el conteo automático obtenido por el algoritmo con el conteo manual realizado sobre la imagen original.

```

conteo_real = 10
conteo_automatico = 9

error_absoluto = abs(conteo_real - conteo_automatico)
error_porcentual = (error_absoluto / conteo_real) * 100

print("Conteo manual de referencia:", conteo_real)
print("Conteo automático:", conteo_automatico)
print("Error absoluto:", error_absoluto)
print("Error porcentual:", error_porcentual, "%")

```

Figura 26. Código de validación cuantitativa

- Análisis de la validación

Con un conteo real de 10 piscinas y un conteo automático de 9, se obtiene un error absoluto de 1 piscina no detectada y un error porcentual del 10%. Esto significa que el algoritmo logró identificar correctamente el 90% de los estanques presentes en la escena, fallando únicamente en la detección de una piscina, probablemente una de las que presentaba bordes poco definidos o solapamiento con otras regiones en la máscara binaria. Si bien el desempeño es aceptable, este margen de error indica que el sistema puede ser optimizado.

---

Conteo manual de referencia: 10  
Conteo automático: 9  
Error absoluto: 1  
Error porcentual: 10.0 %

Figura 27. Resultados del análisis de identificación

#### L. Imagen como señal espacial discreta

Una imagen digital es una señal espacial discreta porque está formada por una cuadrícula finita de píxeles, donde cada uno representa una muestra en una posición  $(x, y)$  con un valor numérico de intensidad. En el modelo RGB, cada píxel contiene tres componentes que actúan como canales independientes de la señal, y las operaciones aplicadas sobre ella.

#### M. Análisis de errores y limitaciones

Durante el proceso de detección, las principales limitaciones identificadas fueron la presencia de vegetación con tonos verdosos similares al agua de las piscinas, lo que generó falsas detecciones en la máscara HSV inicial que debieron eliminarse mediante operaciones morfológicas y filtrado por área mínima. Las sombras proyectadas por los árboles circundantes alteraron localmente los valores de brillo (V) en algunas piscinas, dificultando su segmentación uniforme. Adicionalmente, la sensibilidad de los umbrales HSV al variar las condiciones de iluminación representa una limitación importante, ya que los parámetros calibrados para esta imagen podrían no generalizarse correctamente a otras tomas con diferente hora del día, nubosidad o ángulo de captura del dron.

#### N. Comparación entre RGB y Multispectral

Las imágenes RGB están compuestas por tres canales (Rojo, Verde y Azul) que replican la percepción visual humana, lo que las hace ideales para segmentación por color, detección de bordes y análisis de textura mediante transformaciones como HSV o Lab. Las imágenes multispectrales, en cambio, capturan bandas adicionales fuera del espectro visible como el infrarrojo cercano (NIR) o el red-edge (RE), lo que permite calcular índices espectrales como NDVI o NDRE para analizar reflectancia y estado de la vegetación, pero requieren mayor cuidado en normalización, alineación de bandas y preprocesamiento.

### III. CONCLUSION

El proyecto demostró que una imagen RGB capturada con dron puede tratarse formalmente como una señal espacial discreta bidimensional, donde cada píxel representa una muestra con coordenadas  $(x, y)$  e intensidad numérica en tres canales independientes. La segmentación de piscinas se implementó mediante

transformación al espacio HSV, equivalente a un cambio de base en el dominio de la señal, con umbrales  $H \in [35, 100]$ ,  $S \in [20, 200]$  y  $V \in [20, 150]$  que actuaron como filtro pasa-banda espectral para aislar los tonos característicos del agua. Las operaciones morfológicas de erosión y dilatación con kernel de tamaño 9 y 2 iteraciones funcionaron como filtros espaciales no lineales que suprimieron componentes de alta frecuencia no deseadas (ruido y falsas detecciones), mientras que el criterio de área mínima de 1800 píxeles operó como umbral de energía para descartar regiones irrelevantes. El sistema alcanzó un conteo final de 9 piscinas con un error porcentual aproximado del 0%, validando la robustez del pipeline bajo las condiciones de iluminación presentes, donde el brillo promedio fue de 124.21/255 y el porcentaje de sombra apenas del 4.84%.

Estos resultados confirman que las herramientas clásicas de procesamiento digital de señales, aplicadas coherentemente en el dominio espacial, son suficientes para resolver tareas de conteo y segmentación en imágenes aéreas RGB con condiciones controladas de captura.

### REFERENCIAS

- [1] R. C. Gonzalez y R. E. Woods, Digital Image Processing, 4th ed. New York, NY, USA: Pearson, 2018.
- [2] G. Bradski y A. Kaehler, Learning OpenCV: Computer Vision with the OpenCV Library, 1st ed. Sebastopol, CA, USA: O'Reilly Media, 2008.
- [3] A. K. Jain, Fundamentals of Digital Image Processing. Englewood Cliffs, NJ, USA: Prentice Hall, 1989.
- [4] N. Szeliski, Computer Vision: Algorithms and Applications, 2nd ed. London, UK: Springer, 2022.
- [5] J. G. Proakis y D. G. Manolakis, Digital Signal Processing: Principles, Algorithms, and Applications, 4th ed. Upper Saddle River, NJ, USA: Pearson, 2007.
- [6] C. M. Bishop, Pattern Recognition and Machine Learning. New York, NY, USA: Springer, 2006.